



```
In [1]: import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
```

```
In [2]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Using device: cuda

```
In [3]: transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                          (0.5, 0.5, 0.5)) # Scale to [-1,1]
])
train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=128,
    shuffle=True
)
```

100%|██████████| 170M/170M [00:03<00:00, 47.5MB/s]

```
In [4]: class EncoderDecoder(nn.Module):
    def __init__(self):
        super(EncoderDecoder, self).__init__()
        # Encoder
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1), # 32 -> 16
            nn.ReLU(),
            nn.Conv2d(64, 128, 4, 2, 1), # 16 -> 8
            nn.ReLU(),
            nn.Conv2d(128, 256, 4, 2, 1), # 8 -> 4
            nn.ReLU()
        )
        # Decoder
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(256, 128, 4, 2, 1), # 4 -> 8
            nn.ReLU(),
            nn.ConvTranspose2d(128, 64, 4, 2, 1), # 8 -> 16
            nn.ReLU(),
            nn.ConvTranspose2d(64, 3, 4, 2, 1), # 16 -> 32
            nn.Tanh() # Output in [-1,1]
        )
    def forward(self, x):
```

```

        x = self.encoder(x)
        x = self.decoder(x)
        return x

```

```
In [5]: model = EncoderDecoder().to(device)
```

```
In [6]: criterion = nn.L1Loss()          # Can change to nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.0002)
```

```
In [7]: epochs = 10
for epoch in range(epochs):
    model.train()
    total_loss = 0
    for images, _ in train_loader:
        images = images.to(device)
        outputs = model(images)
        loss = criterion(outputs, images)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch [{epoch+1}/{epochs}] Loss: {total_loss/len(train_loader):.4f}")
print("Training Complete!")
```

```

Epoch [1/10] Loss: 0.1706
Epoch [2/10] Loss: 0.1052
Epoch [3/10] Loss: 0.0889
Epoch [4/10] Loss: 0.0794
Epoch [5/10] Loss: 0.0724
Epoch [6/10] Loss: 0.0672
Epoch [7/10] Loss: 0.0633
Epoch [8/10] Loss: 0.0601
Epoch [9/10] Loss: 0.0575
Epoch [10/10] Loss: 0.0552
Training Complete!

```

```
In [8]: def denormalize(img):
        img = img * 0.5 + 0.5
        return img.clamp(0, 1)
model.eval()
images, _ = next(iter(train_loader))
images = images.to(device)
with torch.no_grad():
    outputs = model(images)
images = denormalize(images)
outputs = denormalize(outputs)
fig, axes = plt.subplots(2, 6, figsize=(8,3))
for i in range(6):
    axes[0,i].imshow(np.transpose(images[i].cpu().numpy(), (1,2,0)),
                     interpolation='nearest')
    axes[0,i].axis('off')
    axes[1,i].imshow(np.transpose(outputs[i].cpu().numpy(), (1,2,0)),
                     interpolation='nearest')
```

```
axes[1,i].axis('off')
axes[0,0].set_title("Original")
axes[1,0].set_title("Reconstructed (Blurry)")
plt.tight_layout()
plt.show()
```

Original



Reconstructed (Blurry)



In []: